TOPIC 4

Stochastic and Mini-batch Gradient Descent: Explained

The original class slides, with a plain explanation under each one

> **NOTE** **How to read this document**
>
> - Every original slide is kept exactly as it was, with its slide number under it
> - Under each slide (or small set of slides) you get a plain explanation: **what you are looking at** and **what it means**
> - The slides come from a recorded class, so a few look like screenshots of a video: that is normal

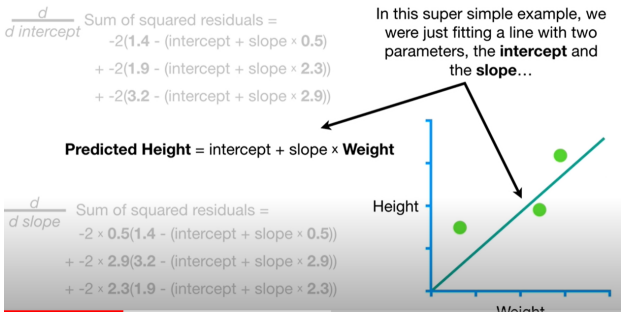
Normal gradient descent looks at **all the data** at every step. That is fine for 3 data points but painfully slow for millions. These 92 slides explain **Stochastic Gradient Descent (SGD)** and **mini-batch gradient descent**: the same idea, but each step only looks at one sample, or a small group of samples.

The same small example from the previous topic is reused: a line with an intercept and a slope, a learning rate of 0.01, and a few green data points. Slides that are almost identical are consecutive animation frames and are explained together.

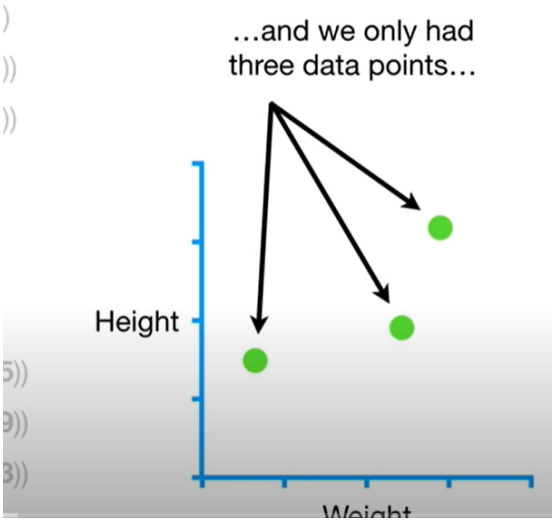
Words you will meet

Word	Plain meaning
Batch gradient descent	Uses all the data at every step
Stochastic gradient descent	Uses 1 random sample per step
Mini-batch	A small random group of samples (for example 3, or 32) used for one step
Parameter	A number the model learns (here the intercept and the slope)
Learning rate	A small number that scales every step; here 0.01
Schedule	How the learning rate changes: large at first, then smaller
Converge	Settle down on a good answer
Redundancy	Many rows of data that are nearly the same, so they carry little new information

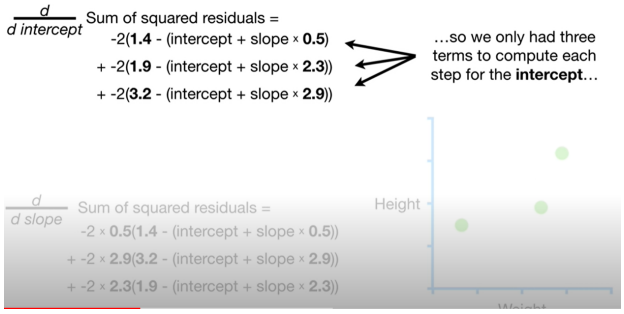
Slides 1 to 5: Recap: our example was tiny



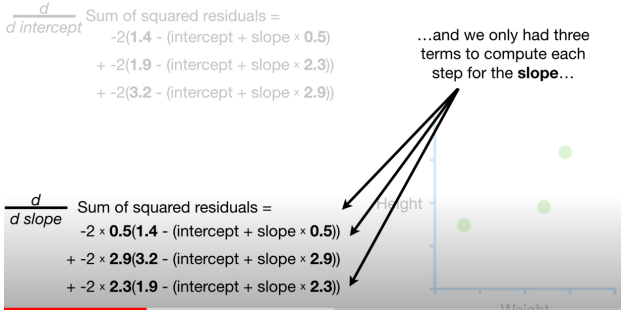
Slide 1



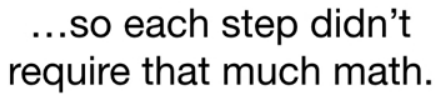
Slide 2



Slide 3



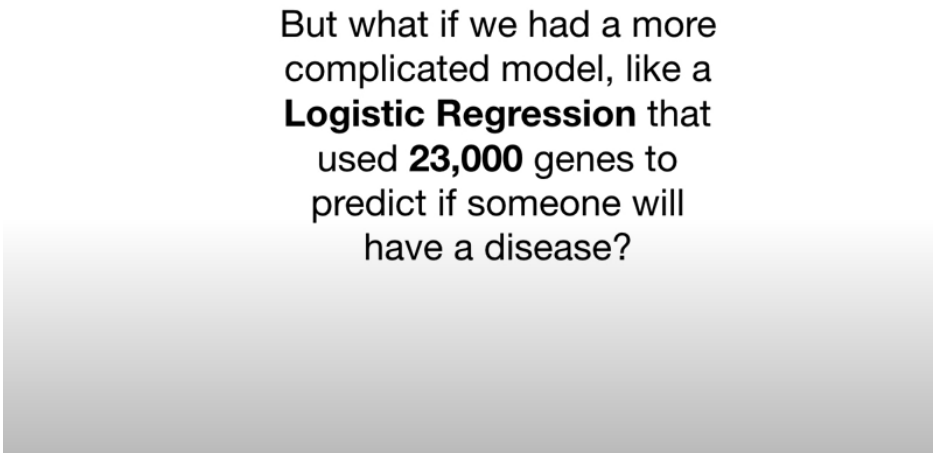
Slide 4



Slide 5

In the earlier example we fitted a line with just **two parameters**, the intercept and the slope, and we had only **three data points**. So each step of gradient descent needed to compute only three terms for the intercept and three for the slope. Each step 'didn't require that much math'.

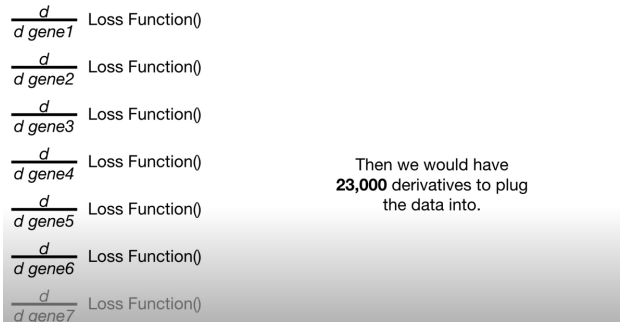
Slide 6: A much bigger model



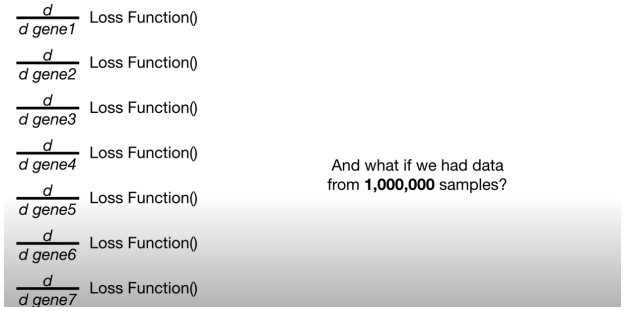
Slide 6

Now imagine a more complicated model: a **logistic regression** that uses **23,000 genes** to predict whether someone will get a disease. Each gene has its own number (parameter) to optimise.

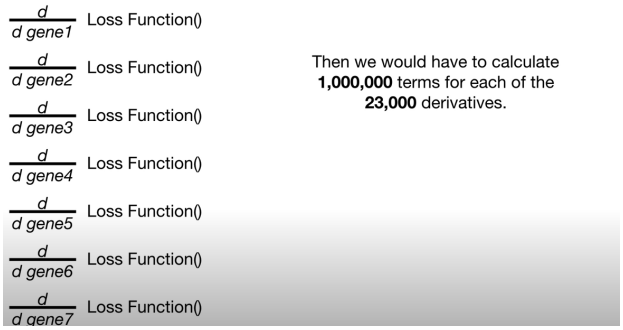
Slides 7 to 9: Counting the work



Slide 7



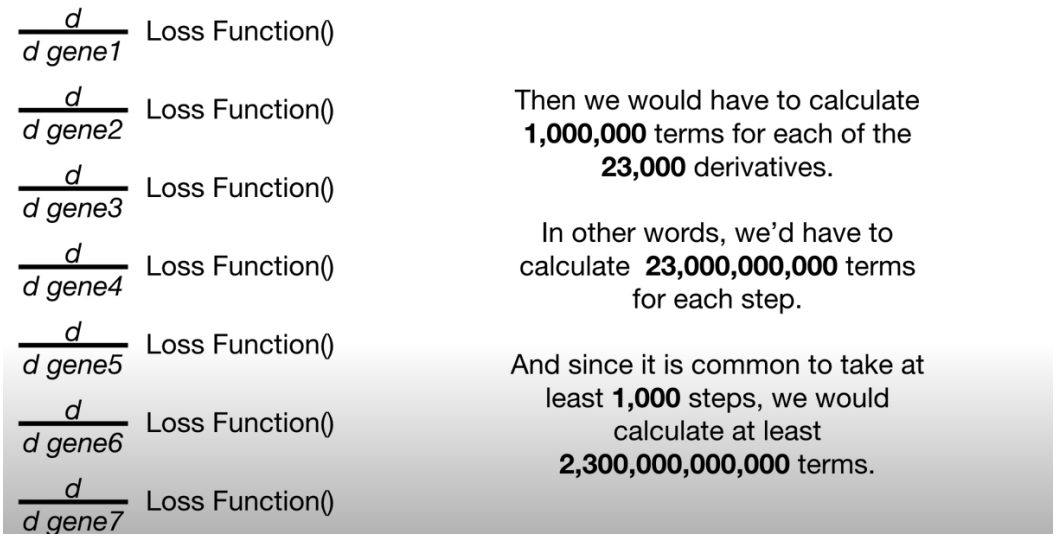
Slide 8



Slide 9

With 23,000 parameters we would need **23,000 derivatives** (one per gene) at every step. And if we also had data from **1,000,000 samples**, then every one of those derivatives needs a term for every sample: **1,000,000 terms for each of the 23,000 derivatives**.

Slide 10: The total work per step and overall

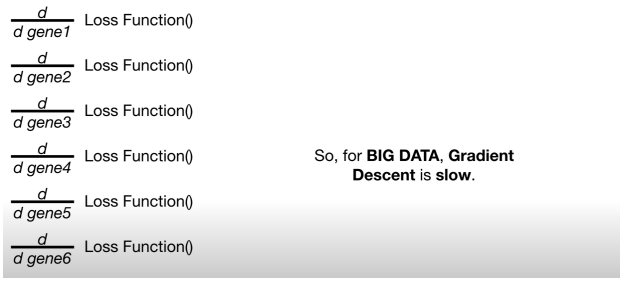


Slide 10

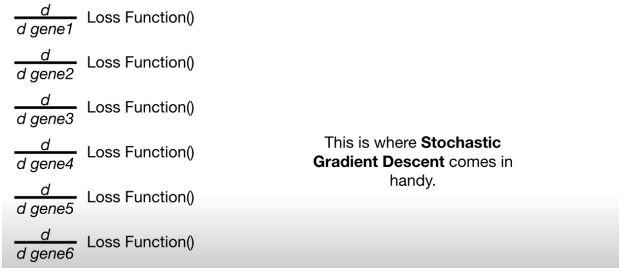
23,000 derivatives × 1,000,000 terms = **23,000,000,000 terms for each step**. Since it is common to take at least **1,000 steps**, the total is 23,000,000,000 × 1,000 terms.

- Note: the slide prints this total as 2,300,000,000,000. The multiplication actually gives 23,000,000,000,000 (23 trillion), so the slide has dropped a zero. The point is the same: it is an enormous amount of arithmetic.
- The slide itself has been kept exactly as it was.

Slides 11 to 12: So for big data, gradient descent is slow



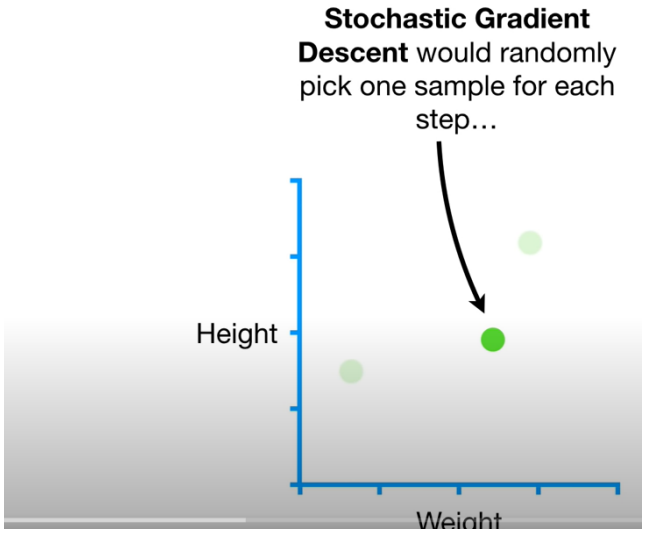
Slide 11



Slide 12

That is why, for **big data, gradient descent is slow**. This is where **Stochastic Gradient Descent** comes in handy.

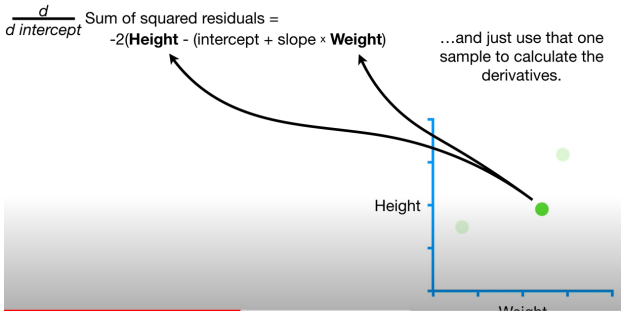
Slide 13: The idea: pick one random sample per step



Slide 13

Stochastic Gradient Descent **randomly picks one sample for each step** (here the bright green dot) and ignores the rest for that step.

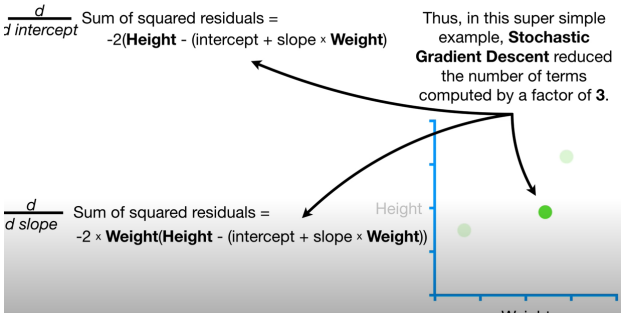
Slides 14 to 16: Only one term per derivative



Slide 14



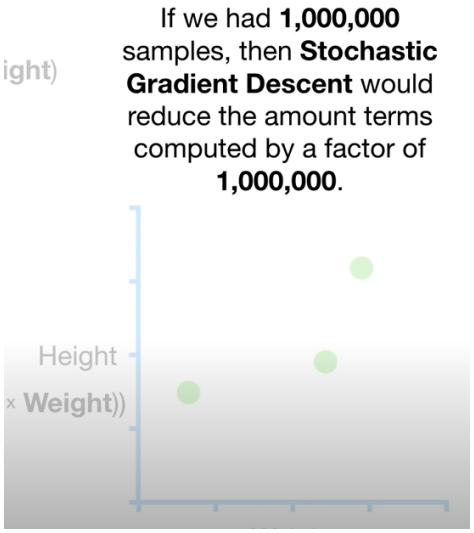
Slide 15



Slide 16

Using just one sample means we only need that one sample to calculate the derivatives for the intercept and for the slope. In this tiny example it **reduced the number of terms computed by a factor of 3** (one term instead of three).

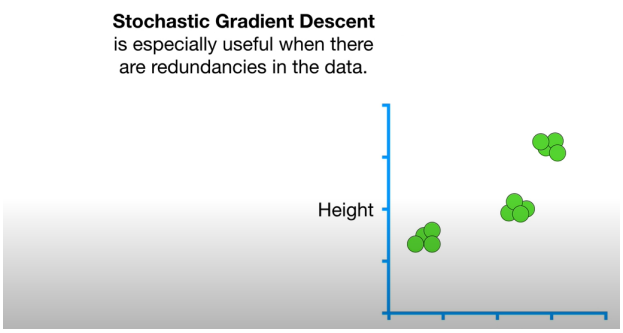
Slide 17: The saving on big data



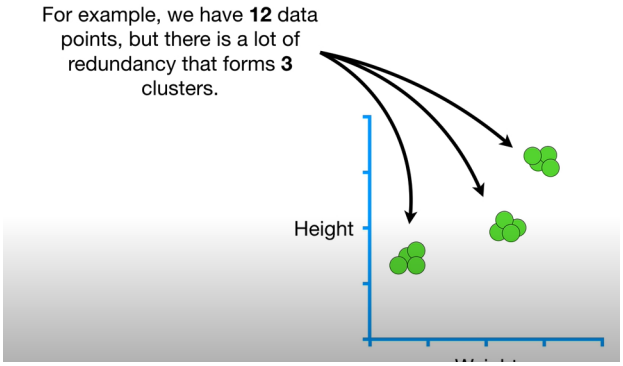
Slide 17

With **1,000,000 samples**, stochastic gradient descent reduces the number of terms computed by a factor of **1,000,000**: each step uses 1 sample instead of 1,000,000.

Slides 18 to 19: When it helps most: redundant data



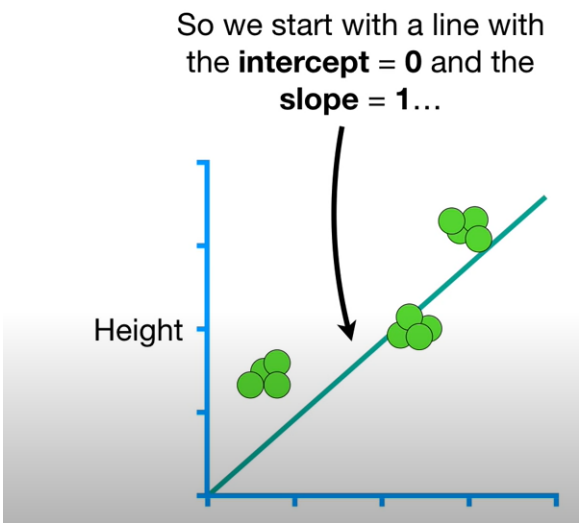
Slide 18



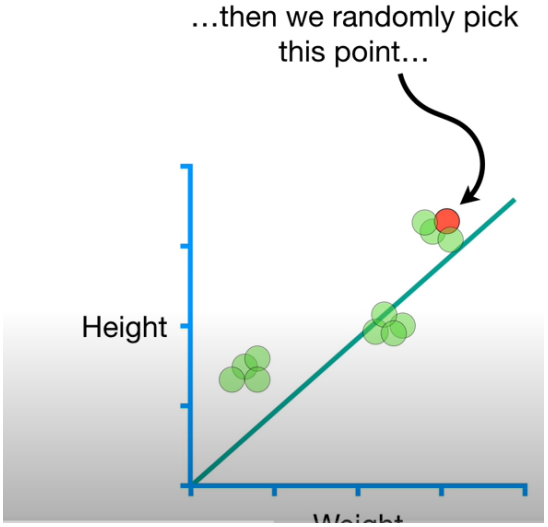
Slide 19

Stochastic gradient descent is **especially useful when there is redundancy in the data**. Here there are 12 data points, but they form just **3 clusters** of 4 similar points. Looking at one point from a cluster tells you almost the same as looking at all four, so most of the work in a full step would be repeated effort.

Slides 20 to 21: A worked example: the starting line



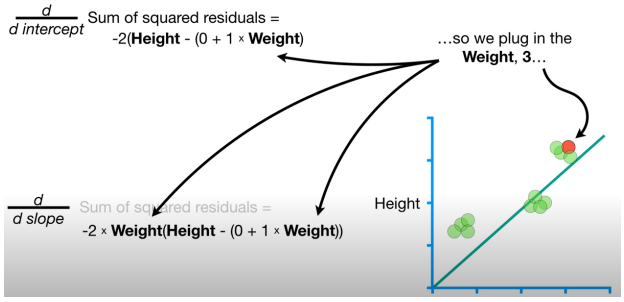
Slide 20



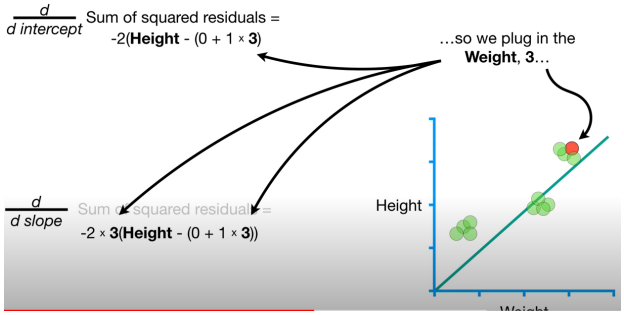
Slide 21

Start with a line that has **intercept = 0 and slope = 1**. Then **randomly pick one point** (the red one in the upper cluster) to decide the first step.

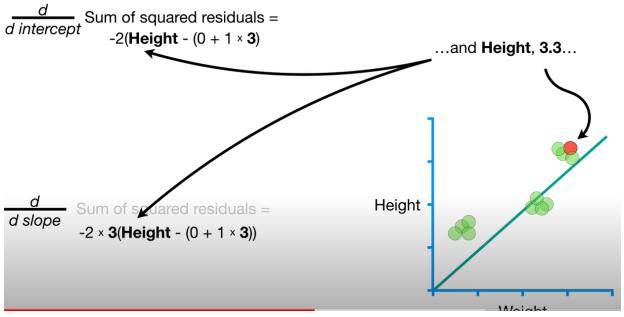
Slides 22 to 26: Plug the picked point into the derivatives



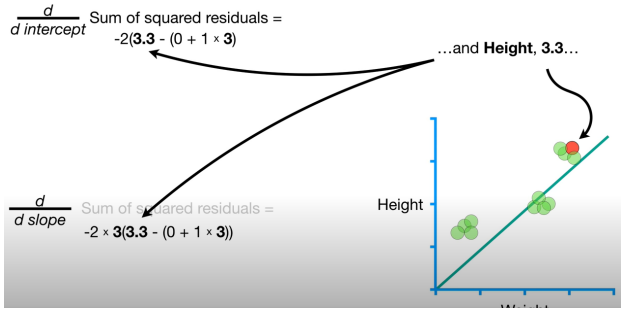
Slide 22



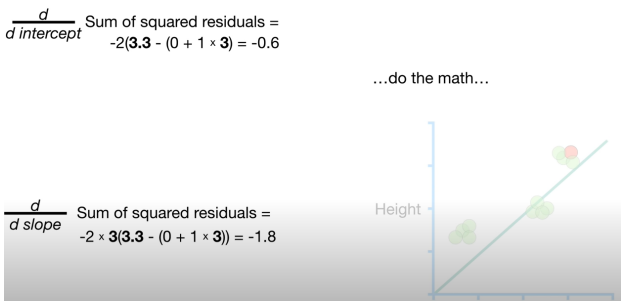
Slide 23



Slide 24



Slide 25



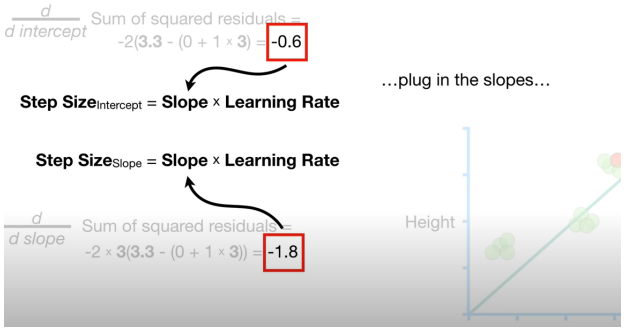
Slide 26

The picked point has **Weight = 3 and Height = 3.3**. Put those numbers into the two derivatives (one for the intercept, one for the slope):

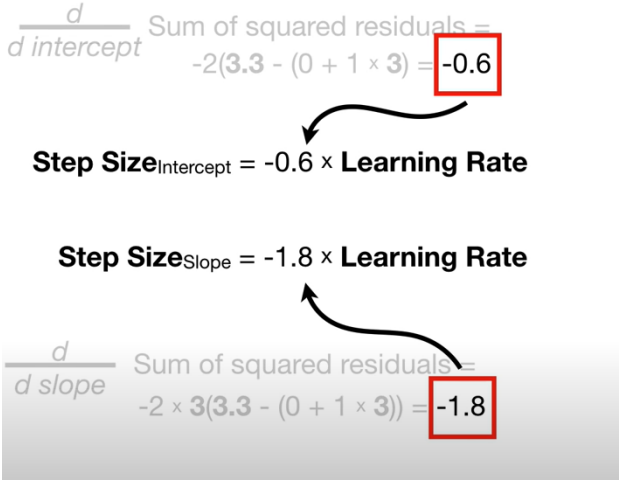
- Intercept derivative: $-2 \times (3.3 - (0 + 1 \times 3)) = \mathbf{-0.6}$
- Slope derivative: $-2 \times 3 \times (3.3 - (0 + 1 \times 3)) = \mathbf{-1.8}$

$$\frac{dL}{d\,\text{intercept}} = -2\,(3.3 - (0 + 1 \times 3)) = -0.6$$

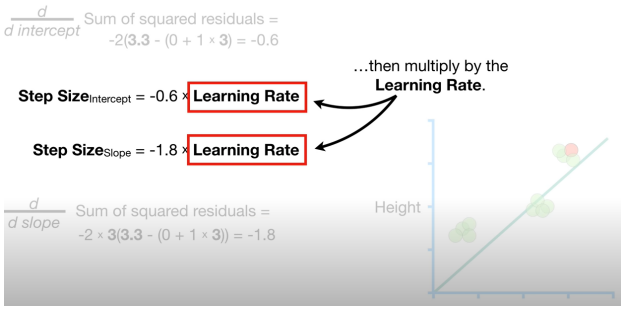
Slides 27 to 29: From slopes to step sizes



Slide 27



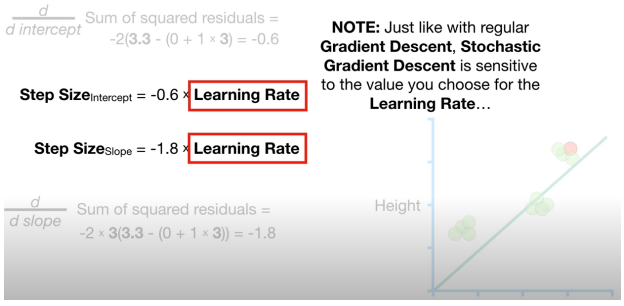
Slide 28



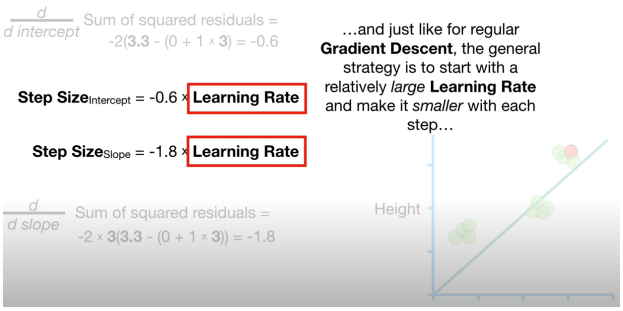
Slide 29

As always, **step size = slope × learning rate**. So the step size for the intercept is $-0.6 \times$ learning rate and the step size for the slope is $-1.8 \times$ learning rate.

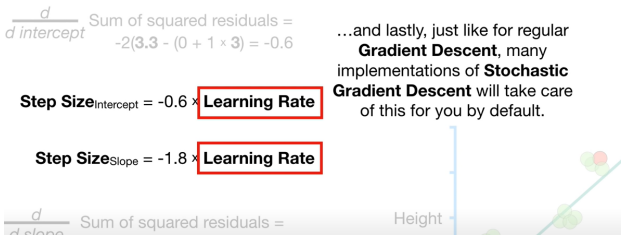
Slides 30 to 32: Choosing the learning rate



Slide 30



Slide 31

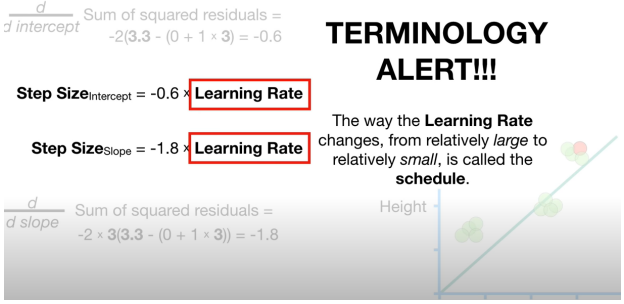


Slide 32

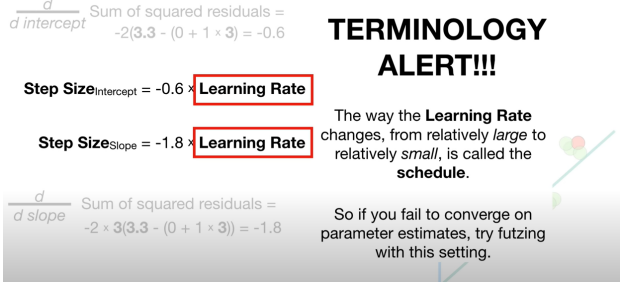
Just like regular gradient descent, stochastic gradient descent is **sensitive to the value you choose for the learning rate**. The usual strategy is to **start with a relatively large learning rate and make it smaller with each step**. Many ready-made implementations of stochastic gradient descent do this for you by default.

- Large rate at first: quick progress when far from the answer
- Smaller rate later: fine adjustments close to the answer

Slides 33 to 34: Terminology: the learning-rate schedule



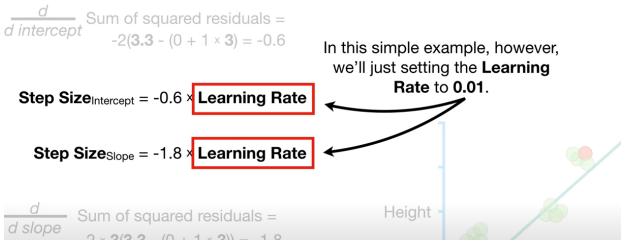
Slide 33



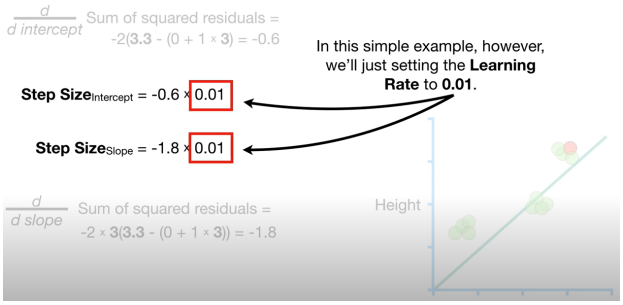
Slide 34

Terminology alert: the way the learning rate changes, from relatively *large* to relatively *small*, is called the **schedule**. If the search fails to **converge** (settle on a good answer), try fiddling with this setting.

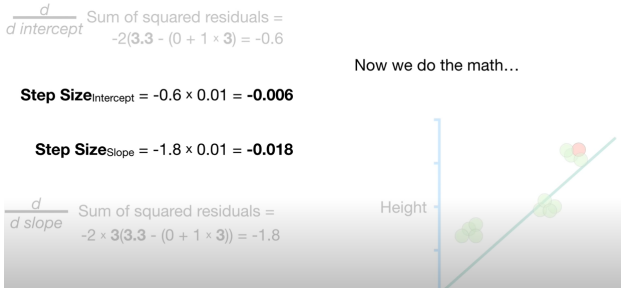
Slides 35 to 37: In the example: learning rate = 0.01



Slide 35



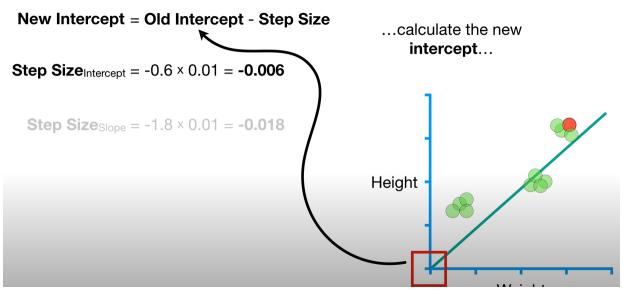
Slide 36



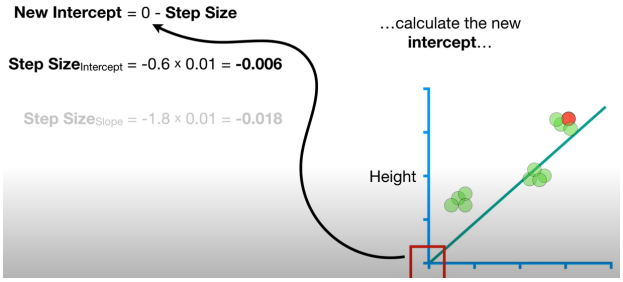
Slide 37

To keep the example simple, the learning rate is just set to **0.01**. So the step sizes are $-0.6 \times 0.01 = \mathbf{-0.006}$ (intercept) and $-1.8 \times 0.01 = \mathbf{-0.018}$ (slope).

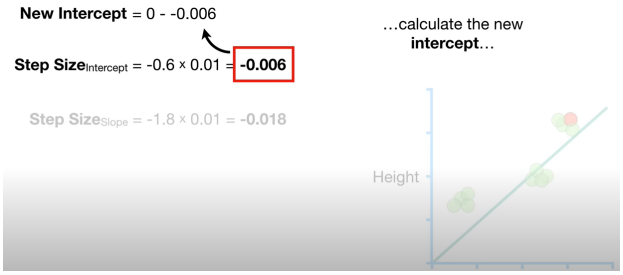
Slides 38 to 41: New intercept



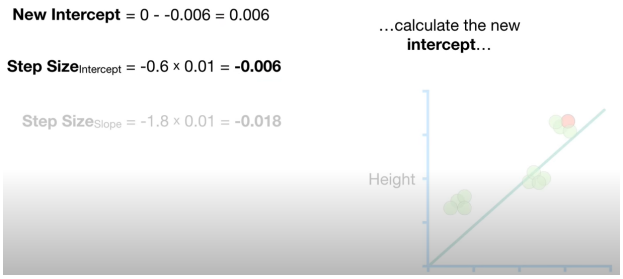
Slide 38



Slide 39



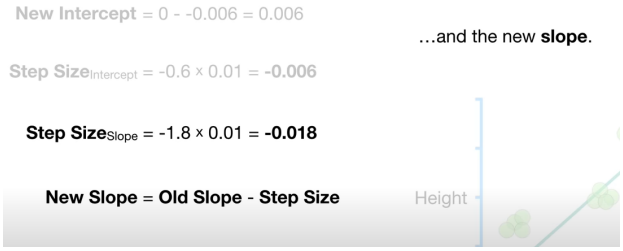
Slide 40



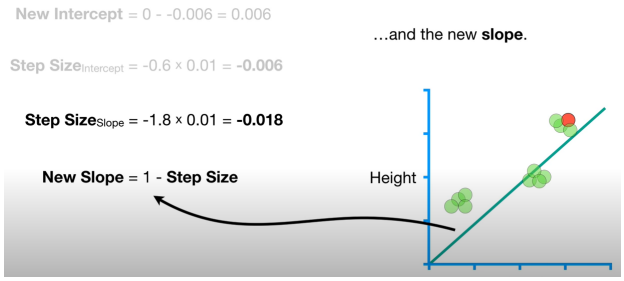
Slide 41

New Intercept = Old Intercept − Step Size = 0 − (−0.006) = 0.006.

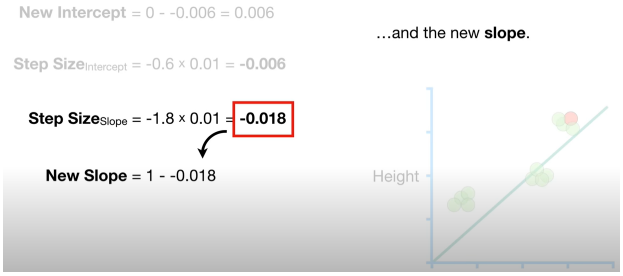
Slides 42 to 46: New slope



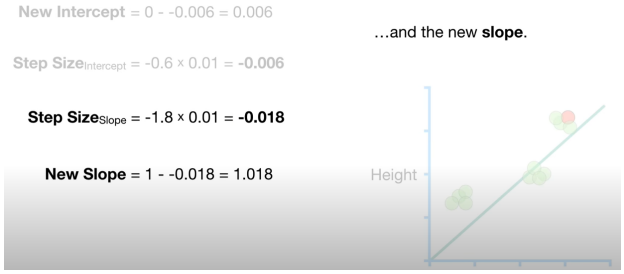
Slide 42



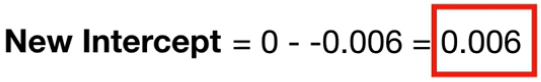
Slide 43

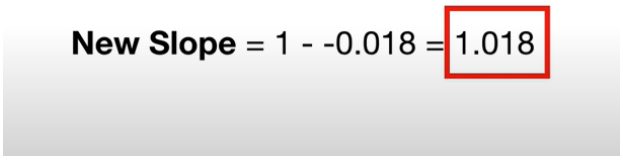


Slide 44



Slide 45



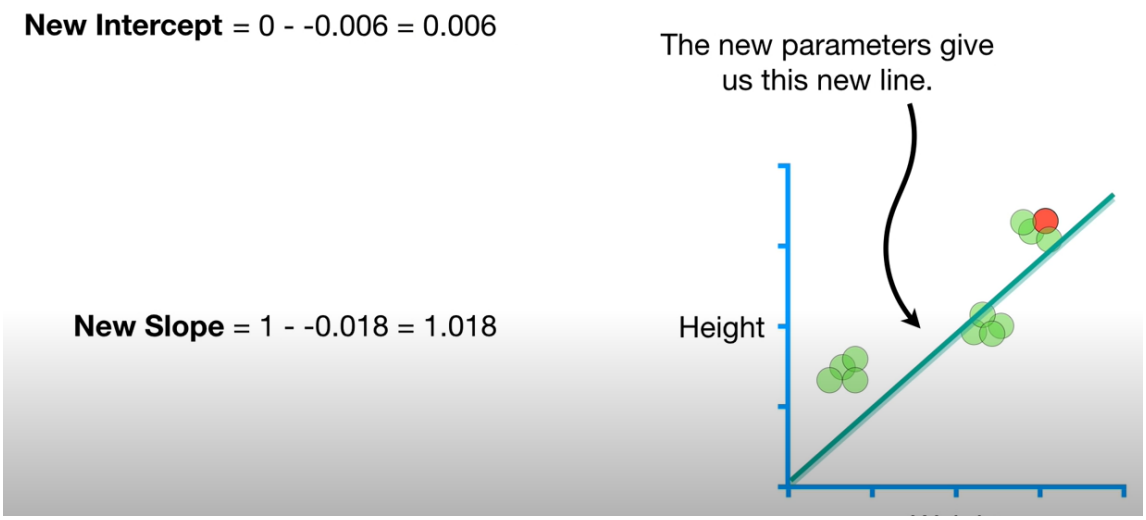


Slide 46

New Slope = Old Slope − Step Size = 1 − (−0.018) = 1.018.

$$\text{New slope} = 1 - (-0.018) = 1.018$$

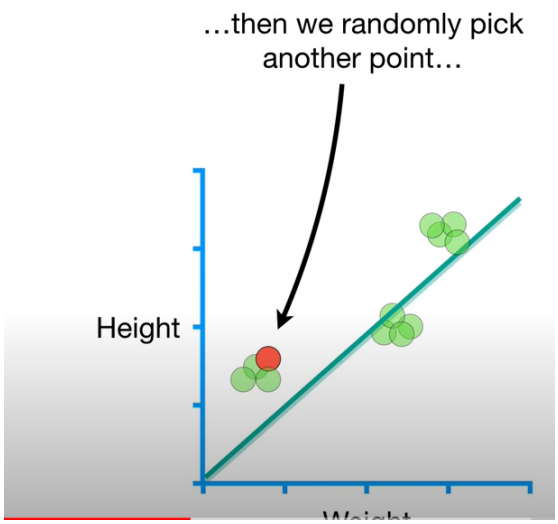
Slide 47: The first update is done



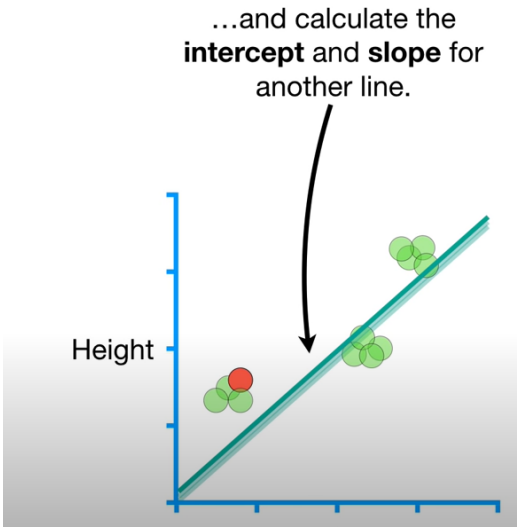
Slide 47

One step of stochastic gradient descent changed the line from (intercept 0, slope 1) to (**intercept 0.006, slope 1.018**). The change is small because the learning rate is small and the single chosen point was already close to the line.

Slides 48 to 49: Repeat with another random point



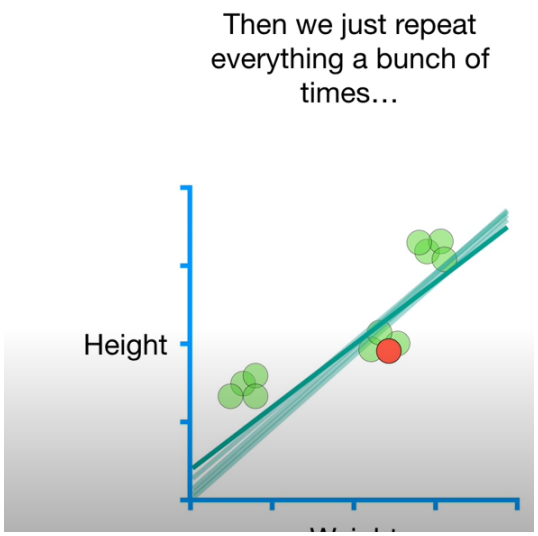
Slide 48



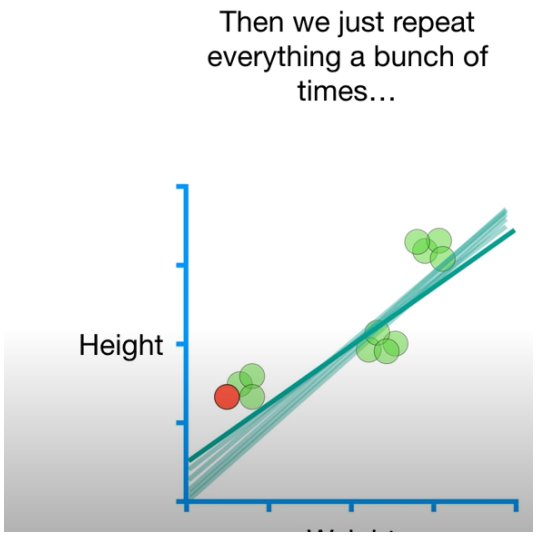
Slide 49

Now **randomly pick another point** (the red dot) and calculate the intercept and slope for another line, using the same recipe.

Slides 50 to 51: Repeat a bunch of times



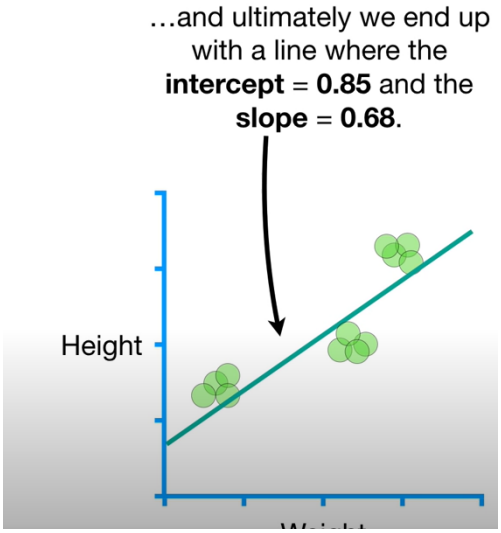
Slide 50



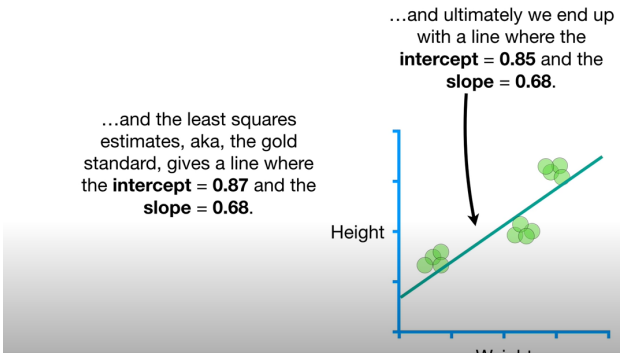
Slide 51

Do the whole thing many times. Each repeat uses a different randomly chosen point and nudges the line a little. The faint teal lines show all the lines the search has tried so far; they gather around the data.

Slides 52 to 53: Result vs the exact answer



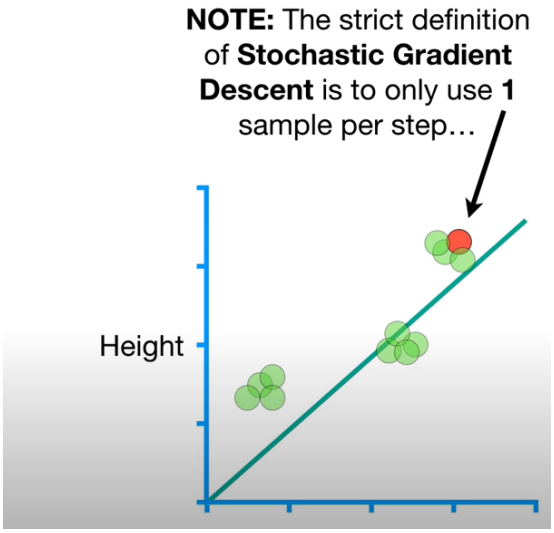
Slide 52



Slide 53

After many steps we end up with a line with **intercept = 0.85 and slope = 0.68**. For comparison, the **least squares** estimate (the 'gold standard', an exact method) gives **intercept = 0.87 and slope = 0.68**. So stochastic gradient descent landed very close to the exact answer while doing far less work per step.

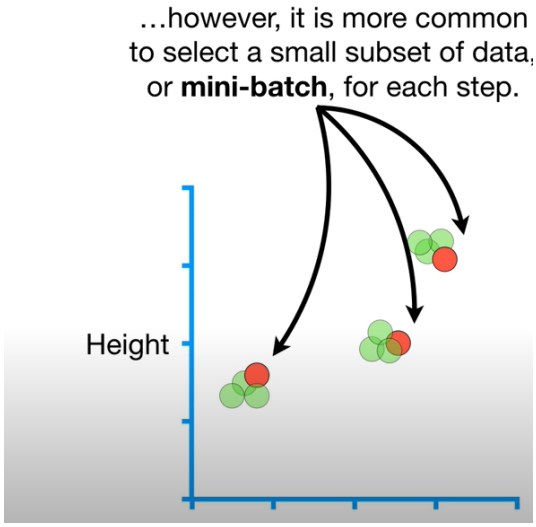
Slide 54: Strict definition: one sample per step



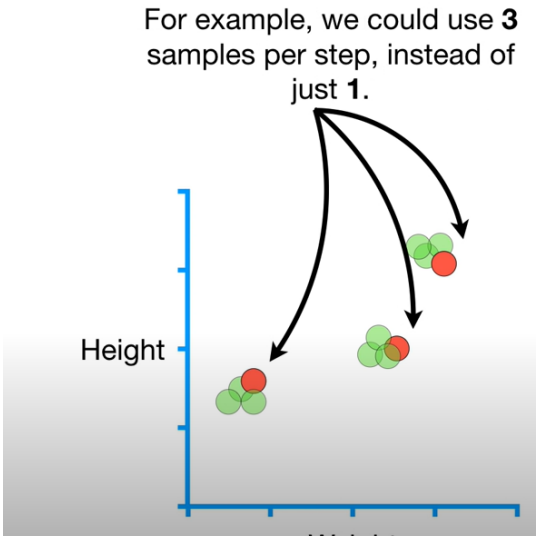
Slide 54

Note: the strict definition of Stochastic Gradient Descent is to use exactly **1 sample per step** (the red dot).

Slides 55 to 56: In practice: mini-batches



Slide 55



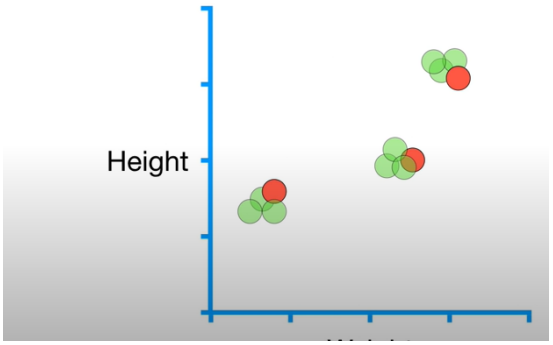
Slide 56

However, it is more common to select a **small subset of the data, called a mini-batch, for each step**. For example we could use **3 samples per step** instead of just 1 (the three red dots, one from each cluster).

- Strict SGD: 1 sample per step
- Mini-batch: a small group of samples per step (for example 3)
- Full (batch) gradient descent: all samples per step

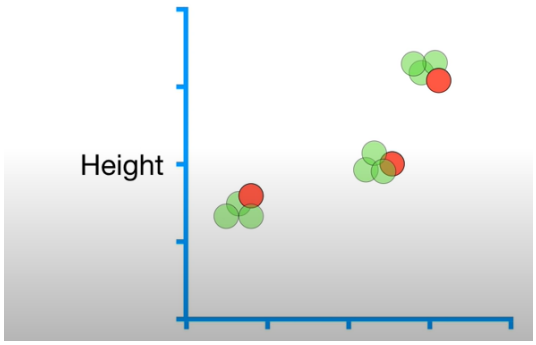
Slides 57 to 59: Mini-batch: the best of both worlds

Using a **mini-batch** for each step takes the best of both worlds between using just one sample and all of of the data at each step.



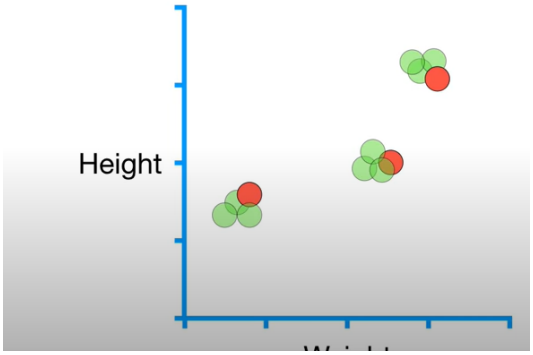
Slide 57

Similar to using all of the data, using a **mini-batch** can result in more stable estimates of the parameters in fewer steps…



Slide 58

…and like using just one sample per step, using a **mini-batch** is much faster than using all of the data.

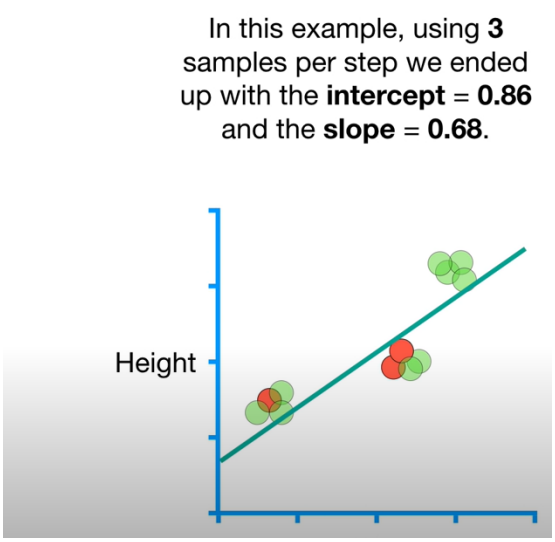


Slide 59

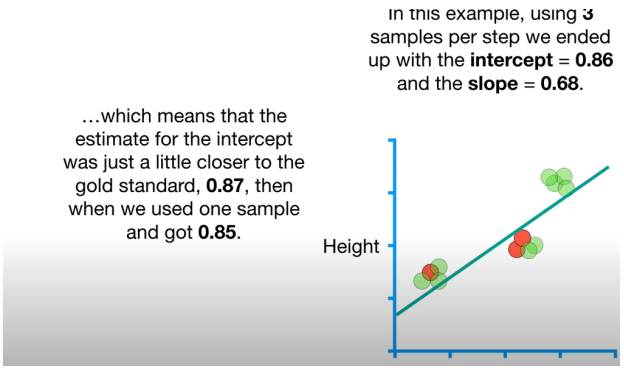
Using a **mini-batch** for each step takes the best of both worlds. Compared with using all the data, it can give **more stable estimates** of the parameters in **fewer steps**. Compared with using just one sample, it is **much faster** than using all of the data.

- One sample: very fast steps, but noisy (the line jumps around)
- All data: smooth and accurate steps, but slow
- Mini-batch: smooth enough and fast enough

Slides 60 to 61: Result with 3 samples per step



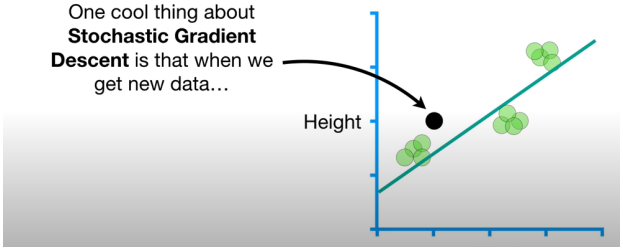
Slide 60



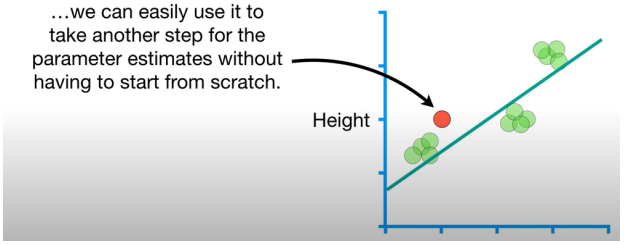
Slide 61

In the example, using 3 samples per step ended with **intercept = 0.86 and slope = 0.68**. The exact (least squares) intercept is **0.87**, so the mini-batch answer (0.86) is a little closer to it than the one-sample answer (0.85).

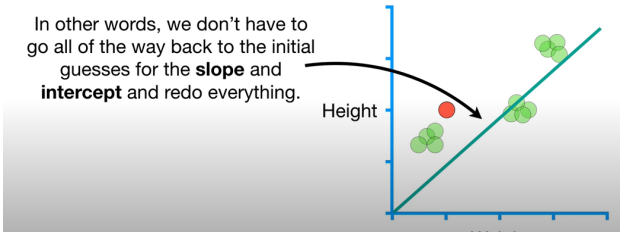
Slides 62 to 65: A bonus: easy to update with new data



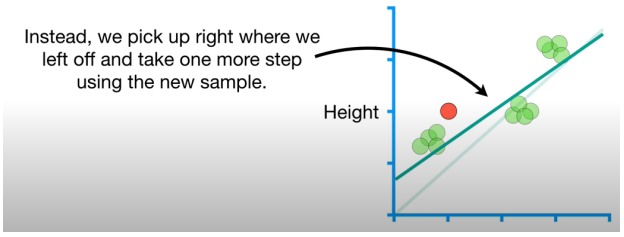
Slide 62



Slide 63



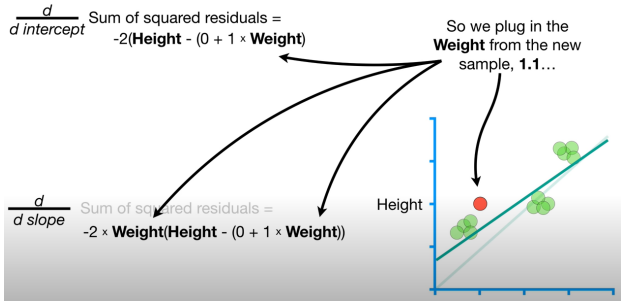
Slide 64



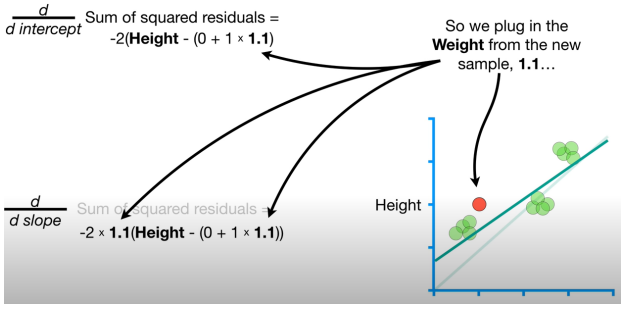
Slide 65

One cool thing about stochastic gradient descent: when we **get new data** (the black dot), we can use it right away to take another step from the current estimates. We do **not** have to go all the way back to the initial guesses for the slope and intercept and redo everything. Instead, we pick up right where we left off and take one more step using the new sample.

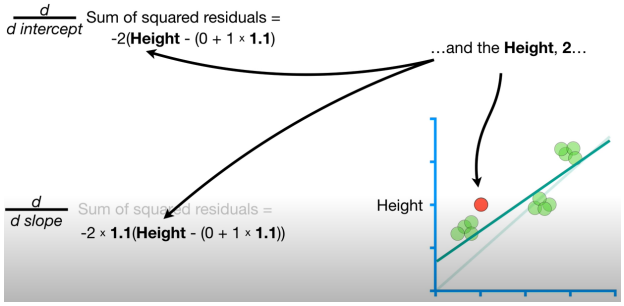
Slides 66 to 70: Worked example: a step with the new sample



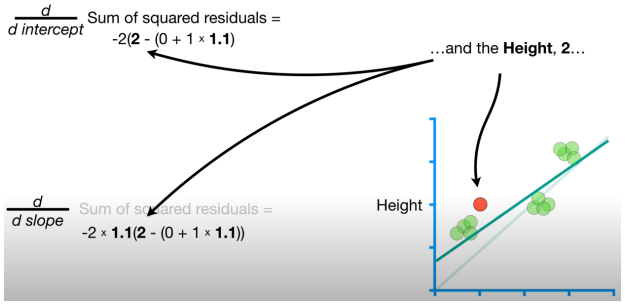
Slide 66



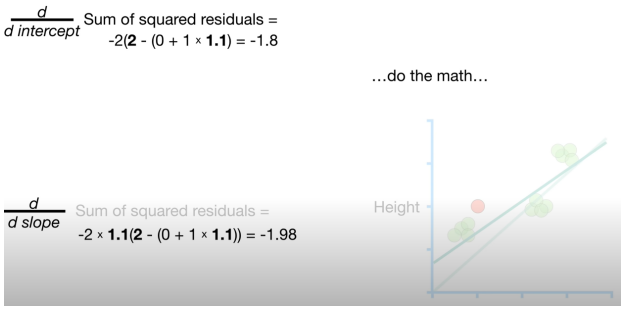
Slide 67



Slide 68



Slide 69

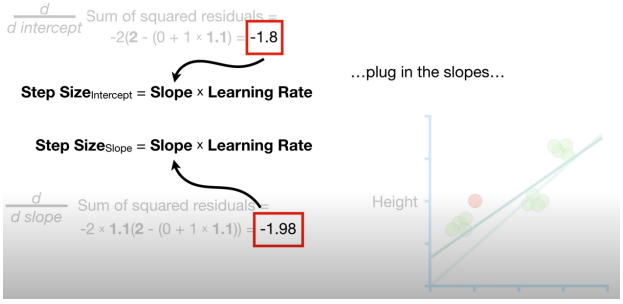


Slide 70

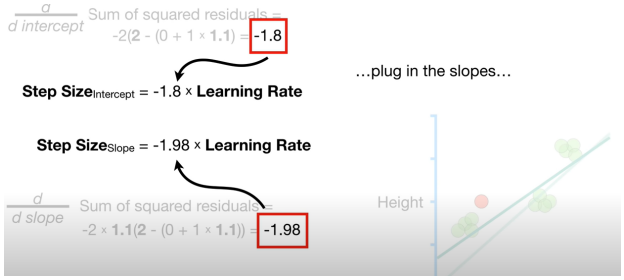
The new sample has **Weight = 1.1 and Height = 2**. Plug these into the two derivatives (the current line is still intercept 0.86, slope 0.68 in the next slides; the derivative formulas are shown with the numbers filled in):

- Derivative for the intercept: $-2 \times (2 - (0 + 1 \times 1.1)) = \mathbf{-1.8}$
- Derivative for the slope: $-2 \times 1.1 \times (2 - (0 + 1 \times 1.1)) = \mathbf{-1.98}$

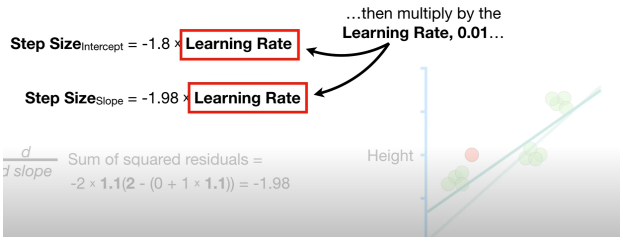
Slides 71 to 76: Step sizes with learning rate 0.01



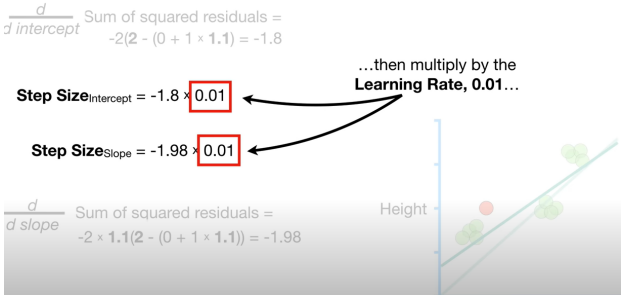
Slide 71



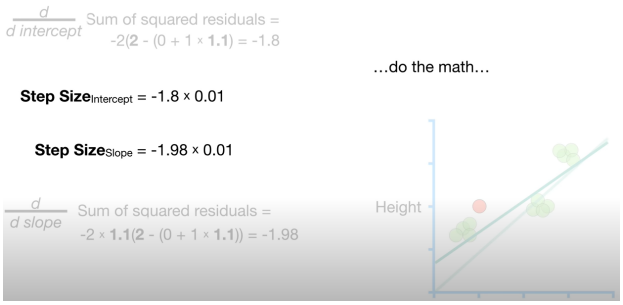
Slide 72



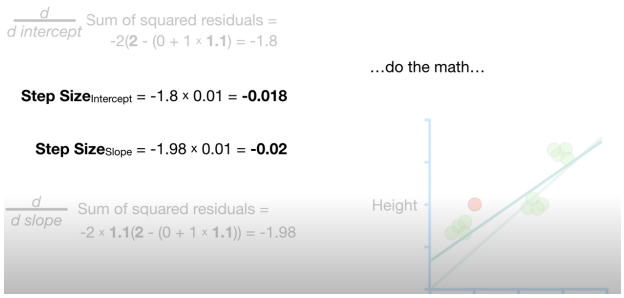
Slide 73



Slide 74



Slide 75

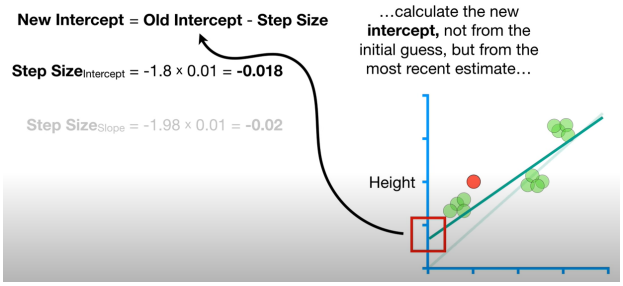


Slide 76

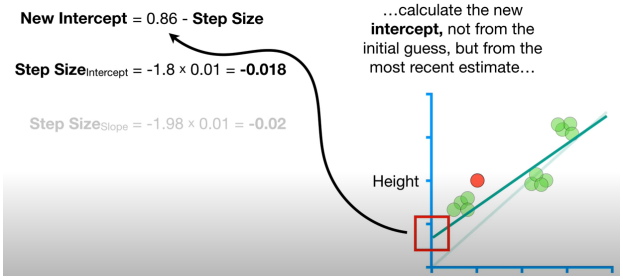
Multiply each slope by the learning rate, **0.01**:

- Step size (intercept) = $-1.8 \times 0.01 = \mathbf{-0.018}$
- Step size (slope) = $-1.98 \times 0.01 = \mathbf{-0.02}$ (rounded from -0.0198)

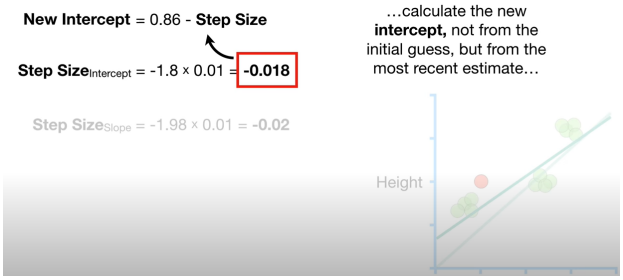
Slides 77 to 81: New intercept, from the most recent estimate



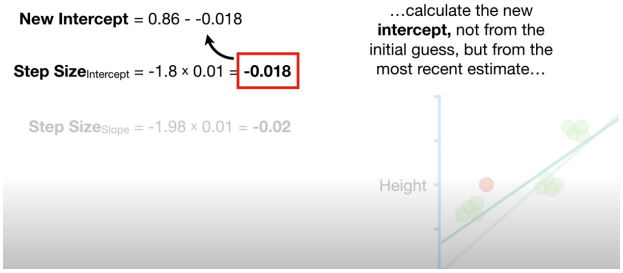
Slide 77



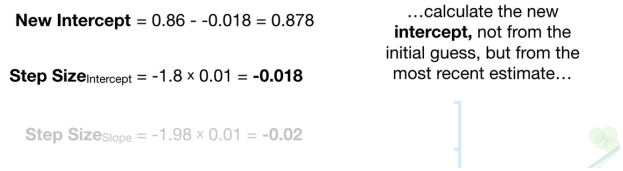
Slide 78



Slide 79



Slide 80

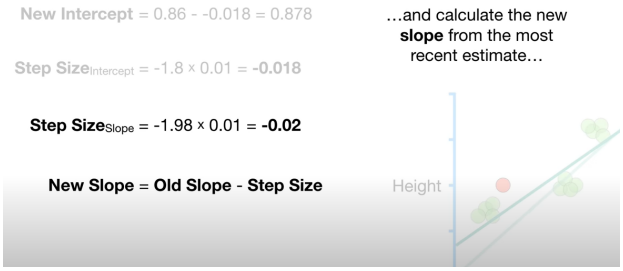


Slide 81

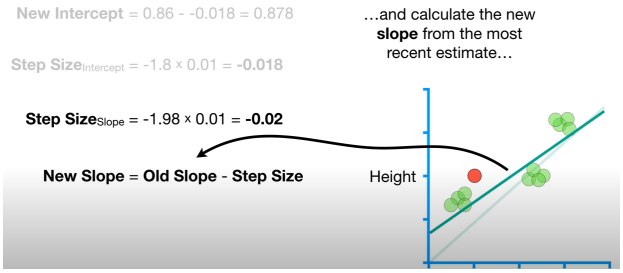
Calculate the new intercept **not from the initial guess but from the most recent estimate**: New intercept = 0.86 − (−0.018) = **0.878**.

$$\text{New intercept} = 0.86 - (-0.018) = 0.878$$

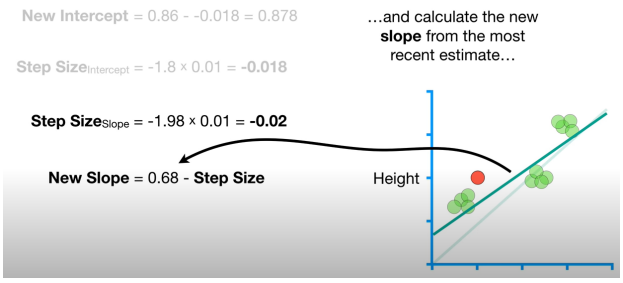
Slides 82 to 85: New slope, from the most recent estimate



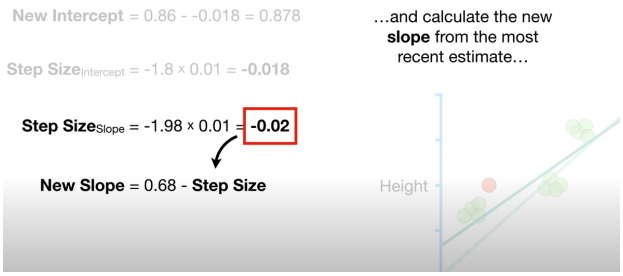
Slide 82



Slide 83



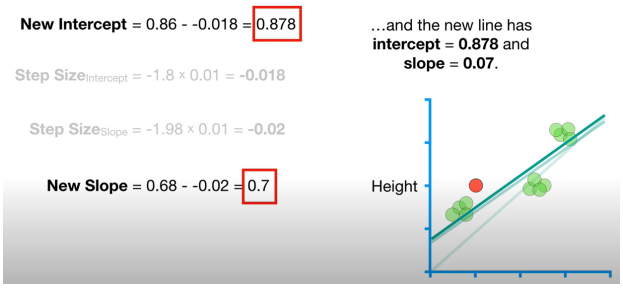
Slide 84



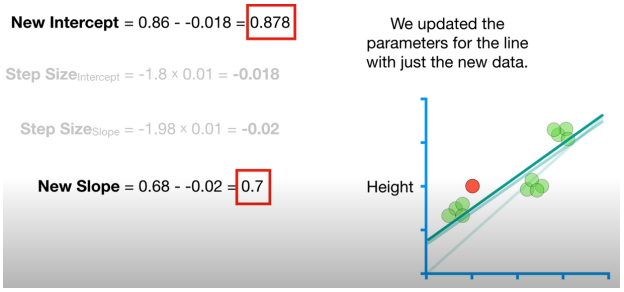
Slide 85

Likewise the new slope is calculated from the most recent slope: New slope = 0.68 − (−0.02) = **0.70**.

Slides 86 to 87: The updated line



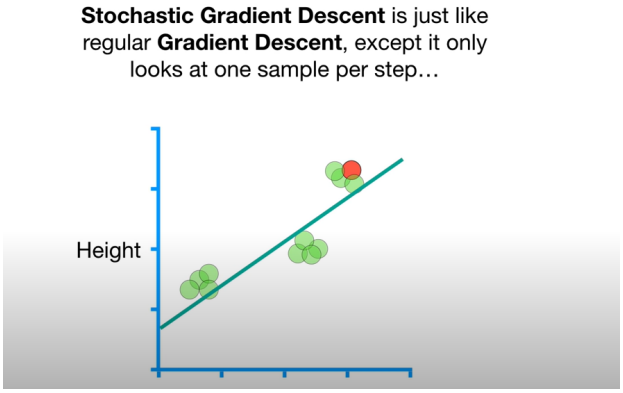
Slide 86



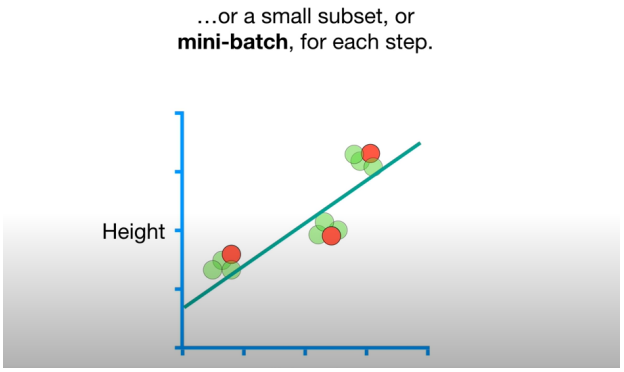
Slide 87

With just the new data point, the parameters have been updated: the new line has **intercept = 0.878 and slope = 0.7**. A single new sample was enough to take one more step.

Slides 88 to 89: Summary of the idea



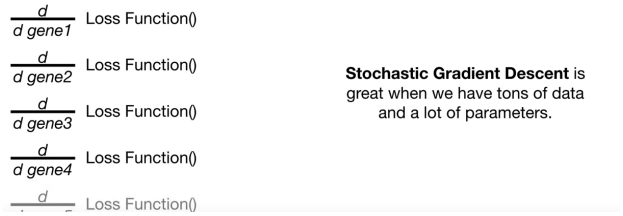
Slide 88



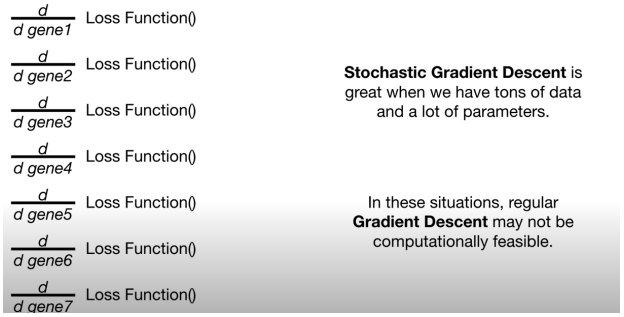
Slide 89

Stochastic Gradient Descent is just like regular gradient descent, except it only looks at one sample per step, or a small subset (a mini-batch) for each step.

Slides 90 to 91: When to use it



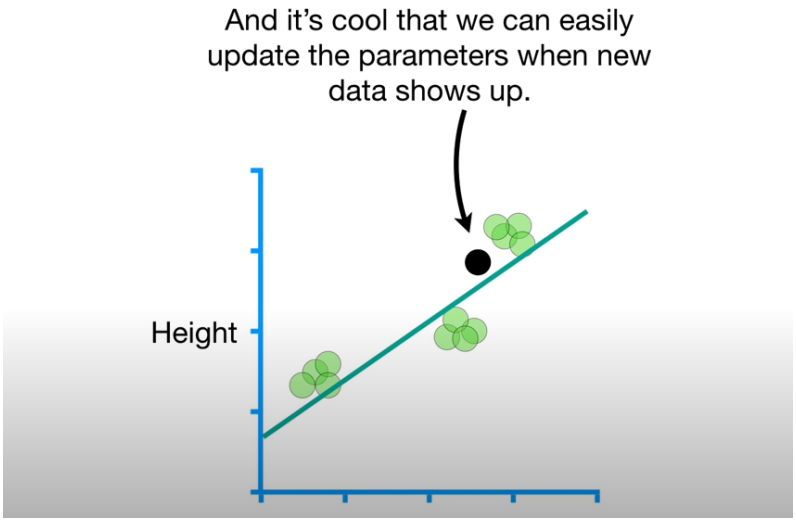
Slide 90



Slide 91

Stochastic gradient descent is **great when we have tons of data and a lot of parameters** (like the 23,000 genes). In those situations, regular gradient descent **may not be computationally feasible**: it would simply take too much work per step.

Slide 92: Final point: easy to keep learning



Slide 92

And it's cool that we can **easily update the parameters when new data shows up** (the black dot): no need to start over.